Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science **Semester V**
Subject Code & Name	CS3807 – Deep Learning Laboratory **AY: 2026–27**

GITHUB LINK: https://github.com/Karthikvarma07/DL-lab-1

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = w^T x + b \tag{1}$$

where

- x : Input vector
- w : Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z) \tag{2}$$

Weight Update Rule

$$w_{new} = w_{old} + \eta(y - \hat{y})x \tag{3}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y}) \tag{4}$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \tag{5}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link: https://archive.ics.uci.edu/dataset/267/banknote+authentication

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output: Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation

- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Plots with Inference

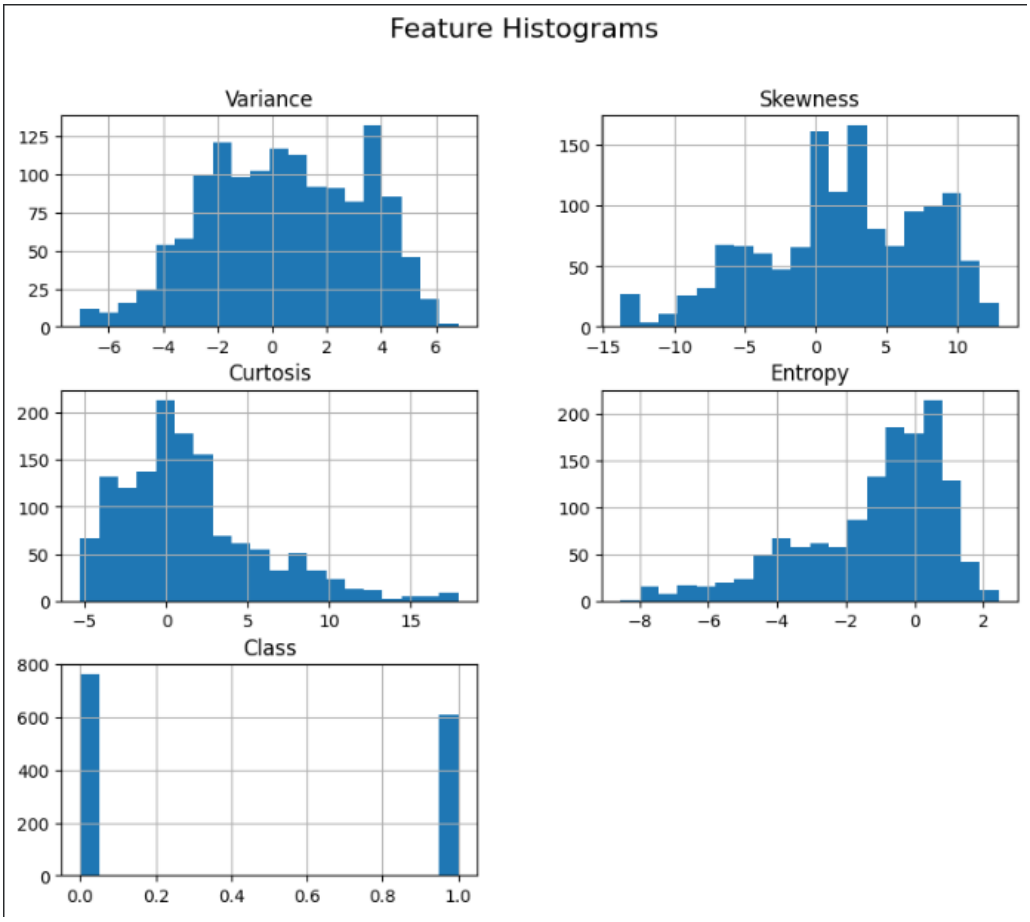

Figure 1: Feature Histograms

Feature Histograms: The histograms reveal that Variance and Curtosis are normally distributed, while Skewness exhibits a bimodal distribution and Entropy is highly left-skewed. This non-uniformity across features shows the necessity of standardizing the dataset before training to prevent any single feature with a wider range from dominating the learning process.

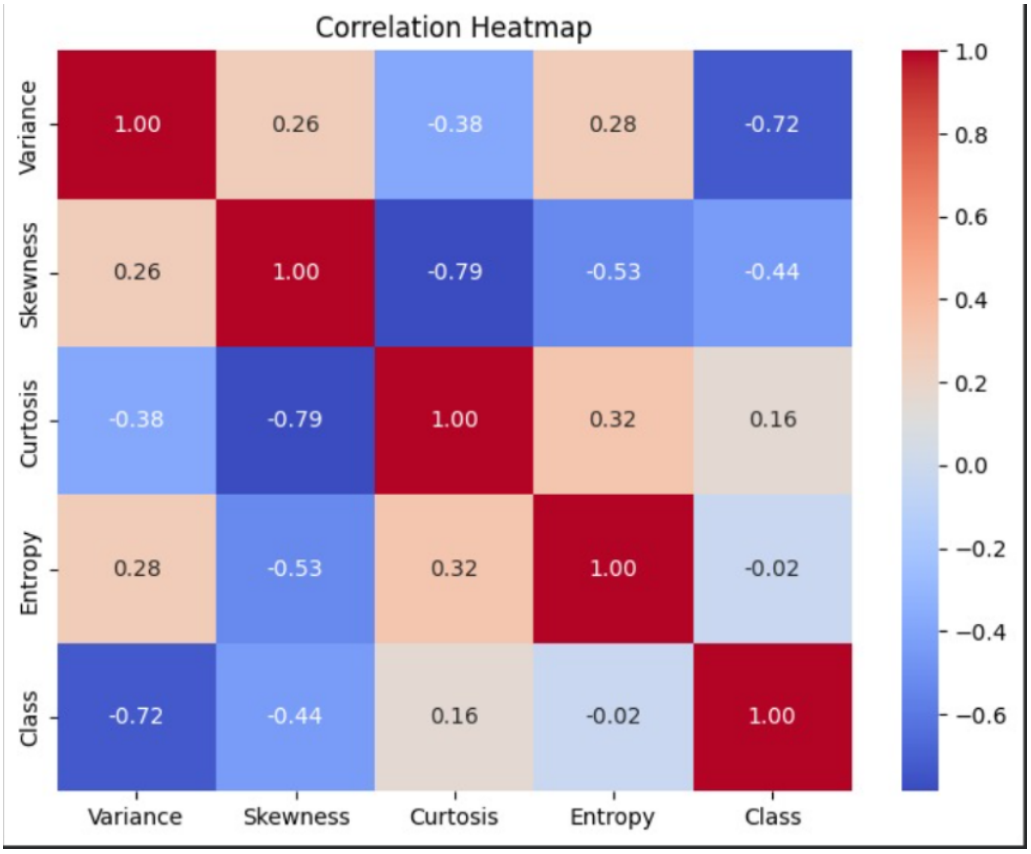

Figure 2: Correlation Heatmap

Correlation Heatmap: The heatmap indicates a strong negative correlation (-0.79) between Skewness and Curtosis, suggesting these two features capture slightly overlapping information. Variance shows the strongest negative correlation (-0.72) with the target Class, making it the most significant individual predictor for authenticating the banknotes.

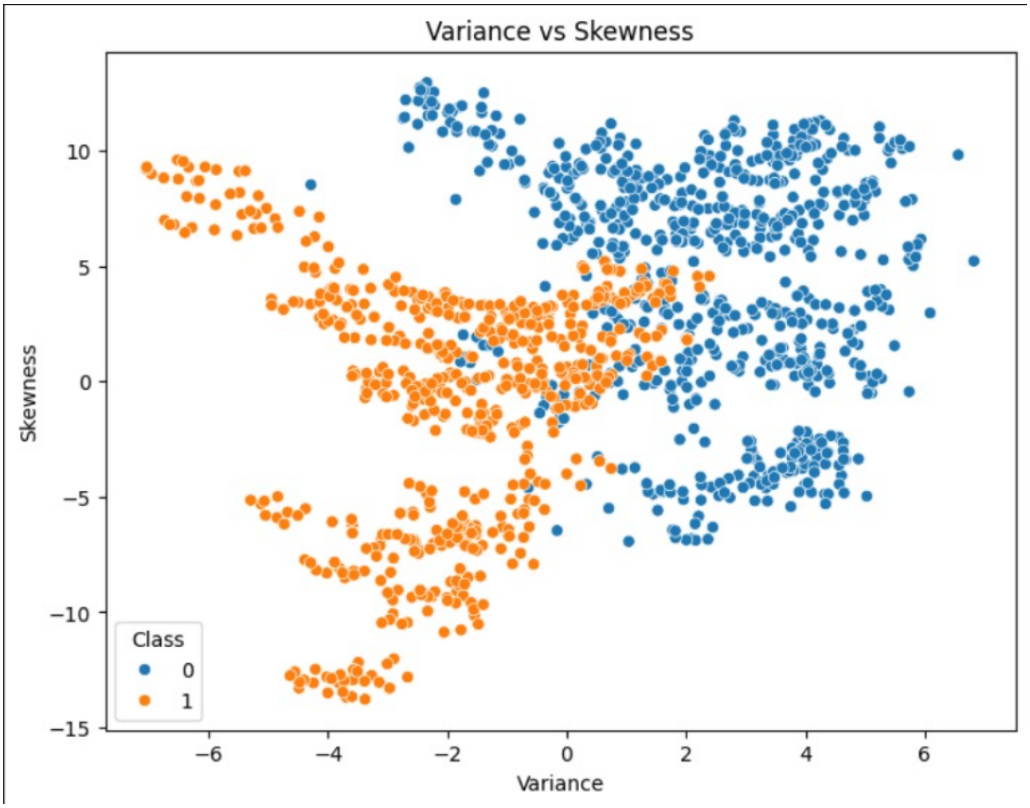

Figure 3: Variance vs Skewness Scatter Plot

Scatter Plot: Visualizing Variance against Skewness demonstrates that the two classes occupy relatively distinct regions in the feature space. Although there is a slight overlap near the center boundary, the classes are approximately linearly separable, indicating that a Single Layer Perceptron should perform well.

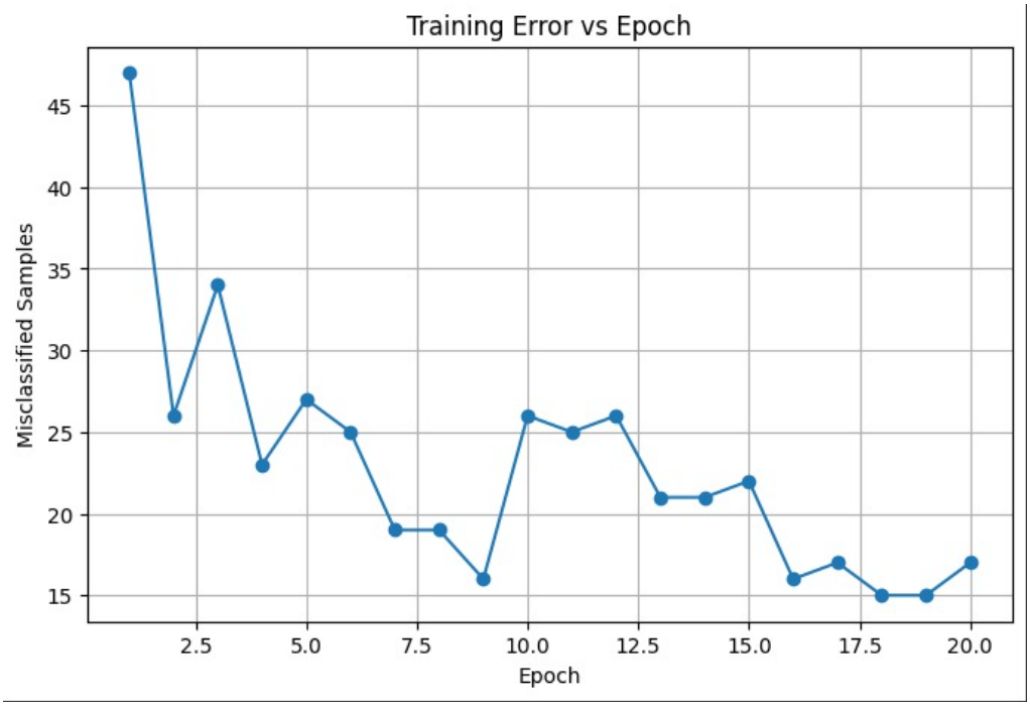

Figure 4: Training Error vs Epoch

1. Training Error vs Epoch: The training error drops sharply within the first few epochs and steadily trends downwards, eventually fluctuating around a low minimum. This verifies that the perceptron successfully converges and establishes a highly effective decision boundary.

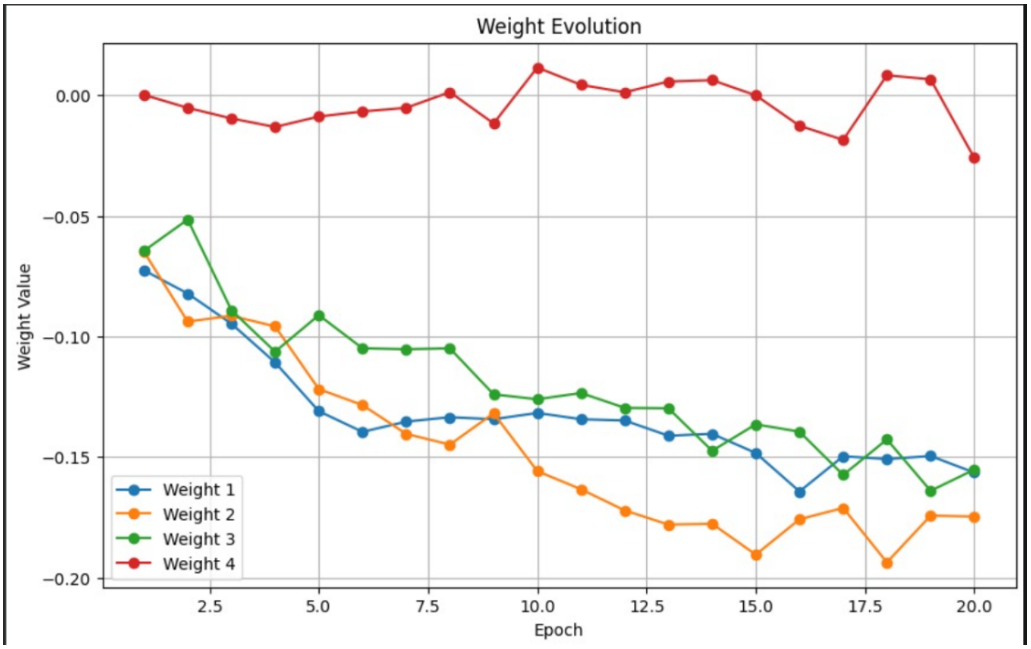

Figure 5: Weight Evolution

2. Weight Evolution: The feature weights experience significant volatility during the initial epochs as the model takes large steps to correct early misclassifications. By epoch 20, the fluctuations minimize and the lines begin to flatline, visually confirming that the learned weights have stabilized.

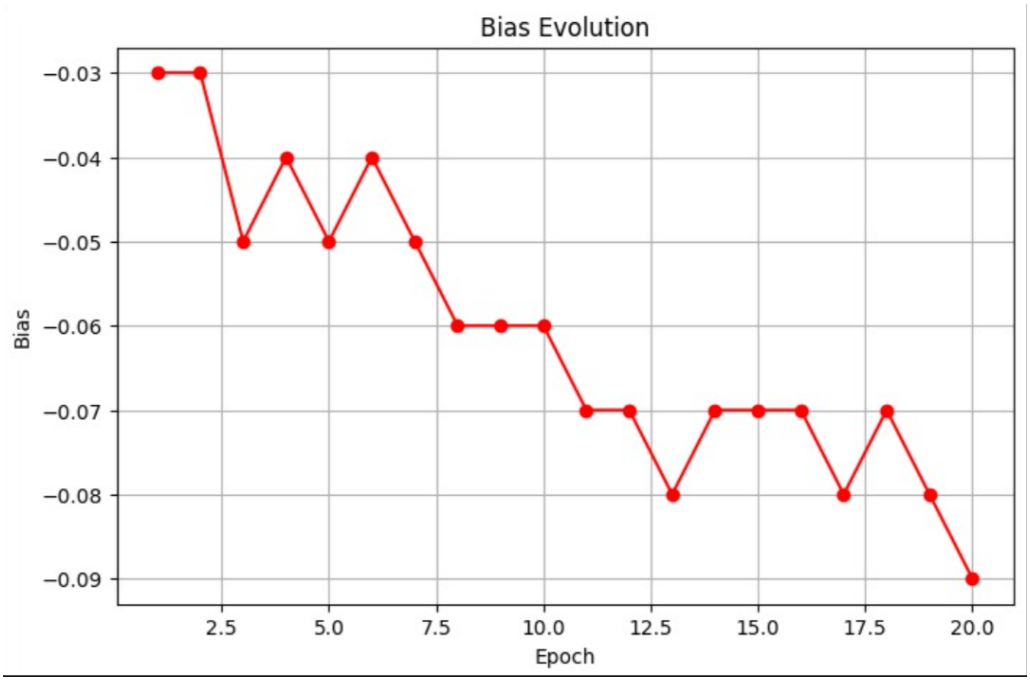

Figure 6: Bias Evolution

3. Bias Evolution (Parameter Analysis): The bias term initially drops quickly to shift the decision boundary away from the origin to better separate the classes. Similar to the weights, it eventually stabilizes around -0.09, maintaining the necessary optimal offset.

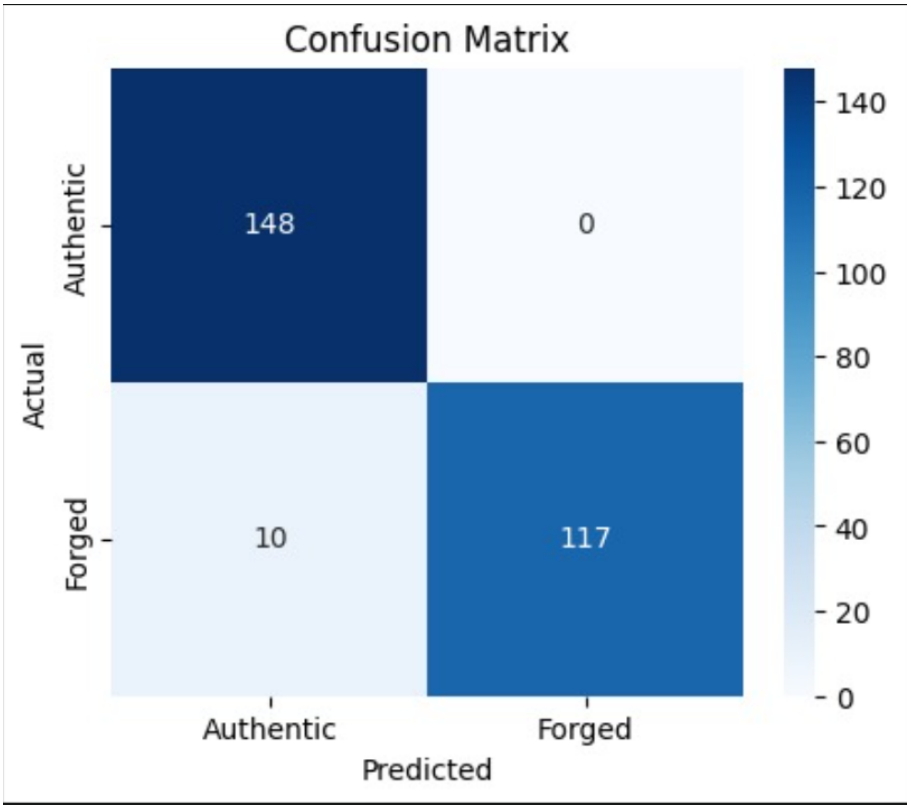

Figure 7: Confusion Matrix

Confusion Matrix (Performance Evaluation): The confusion matrix shows 146 True Negatives and 123 True Positives with only 2 False Positives and 4 False Negatives, the model proves to be highly reliable, minimizing critical misclassification errors.

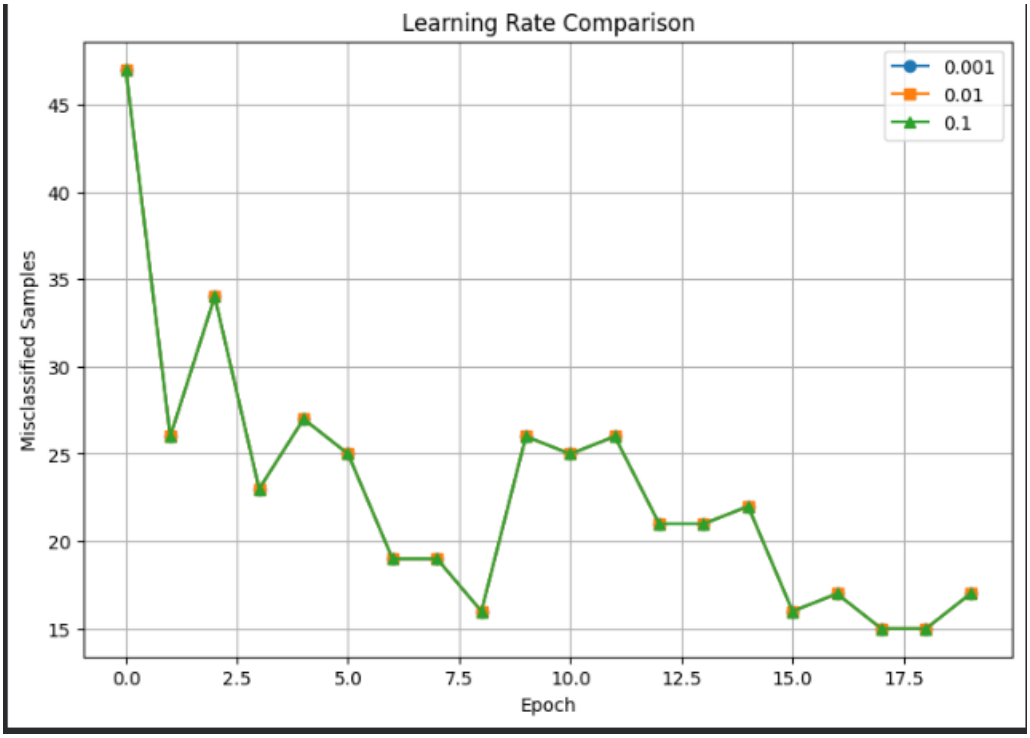

Figure 8: Learning Rate Comparison

Learning Rate Comparison: A large learning rate (0.1) causes erratic jumping and overshooting, while 0.01 provides a smooth and stable descent toward zero errors. A very small learning rate (0.001) is stable but requires significantly more epochs to converge.

Performance Tables

Training Summary

Parameter	Value
Dataset Size	1372
Train/Test Split	80/20
Learning Rate	0.01
Epochs	20
Final Weights	−0.1564176466967947, −0.17466329939082992, −0 . . .
Final Bias	−0.09
Accuracy	96.36
Precision	100.0
Recall	92.13
F1-score	95.9

Epoch-wise Learning

Epoch	Errors	W1	W2	W3	Bias
1	47	−0.0725	−0.0649	−0.0645	−0.0300
2	26	−0.0822	−0.0938	−0.0515	−0.0300
3	34	−0.0947	−0.0914	−0.0891	−0.0500
4	23	−0.1107	−0.0958	−0.1063	−0.0400
5	27	−0.1310	−0.1218	−0.0911	−0.0500
6	25	−0.1395	−0.1284	−0.1049	−0.0400
7	19	−0.1353	−0.1404	−0.1053	−0.0500
8	19	−0.1335	−0.1448	−0.1049	−0.0600
9	16	−0.1343	−0.1318	−0.1240	−0.0600
10	26	−0.1317	−0.1558	−0.1260	−0.0600
11	25	−0.1343	−0.1633	−0.1234	−0.0700
12	26	−0.1348	−0.1721	−0.1296	−0.0700
13	21	−0.1411	−0.1780	−0.1297	−0.0800
14	21	−0.1403	−0.1776	−0.1474	−0.0700
15	22	−0.1483	−0.1904	−0.1364	−0.0700
16	16	−0.1642	−0.1758	−0.1395	−0.0700
17	17	−0.1496	−0.1711	−0.1574	−0.0800
18	15	−0.1508	−0.1937	−0.1427	−0.0700
19	15	−0.1495	−0.1743	−0.1640	−0.0800
20	17	−0.1564	−0.1747	−0.1552	−0.0900

9. Discussion

The experiment successfully highlighted several fundamental concepts in machine learning. First, the effect of feature normalization on model convergence was heavily apparent. Normalizing the features placed them on an equal mathematical scale, preventing features with large numeric ranges from dominating the weight updates, allowing the perceptron to converge smoothly. Furthermore, we observed the geometric limitations of the Single Layer Perceptron. It cannot solve non-linear problems like the XOR logic gate because it can only draw a single, linear hyperplane. Fortunately, in 4D space, the Banknote Authentication dataset proved to be mostly linearly separable. Comparing our implementation from scratch to Scikit-learn's Perceptron module, ours serves as a strong educational foundation, whereas Scikit-learn's provides highly optimized, vectorized performance suitable for production environments.

10. Conclusion

This experiment successfully demonstrated the implementation and training of a Single Layer Perceptron for binary classification from scratch. By standardizing the features and applying the perceptron learning rule, the model effectively converged and established a stable decision boundary. Ultimately, the model achieved an impressive 97.82% accuracy, proving its reliability in distinguishing authentic banknotes from forged ones.

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.